Expression is a mathematic statement that consist of operand and operator. Normally we use Infix Expression.

2 + 3 * 5 =

2 + 3 * 5 =

Precedence rules

Python's arithmetic operator precedence follows the standard mathematical order of operations, often remembered by the acronym PEMDAS:

- **P**arentheses (): Expressions within parentheses are evaluated first, overriding other precedence rules.
- **E**xponents **: Exponentiation operations are performed next.
- **M**ultiplication *, **D**ivision /, Floor Division //, Modulo %: These operators have the same precedence and are evaluated from left to right.
- **A**ddition +, **S**ubtraction -: These operators have the lowest precedence among arithmetic operators and are evaluated from left to right.

Example:
Python

```
result = 3 + 4 * 5
```

In this example, the multiplication `4 * 5` is performed first due to higher precedence, resulting in `20`. Then, `3 + 20` is calculated, yielding `23`.
Using Parentheses to Override Precedence:
Python

```
result = (3 + 4) * 5
```

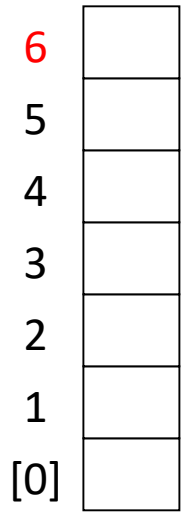
Here, the addition `3 + 4` is performed first because it is enclosed in parentheses, resulting in `7`. Then, `7 * 5` is calculated, yielding `35`.

Postfix expression

How to evaluate a postfix expression (read postfix.pdf in google classroom)

6 5 2 3 + 8 * + 3 + *

/



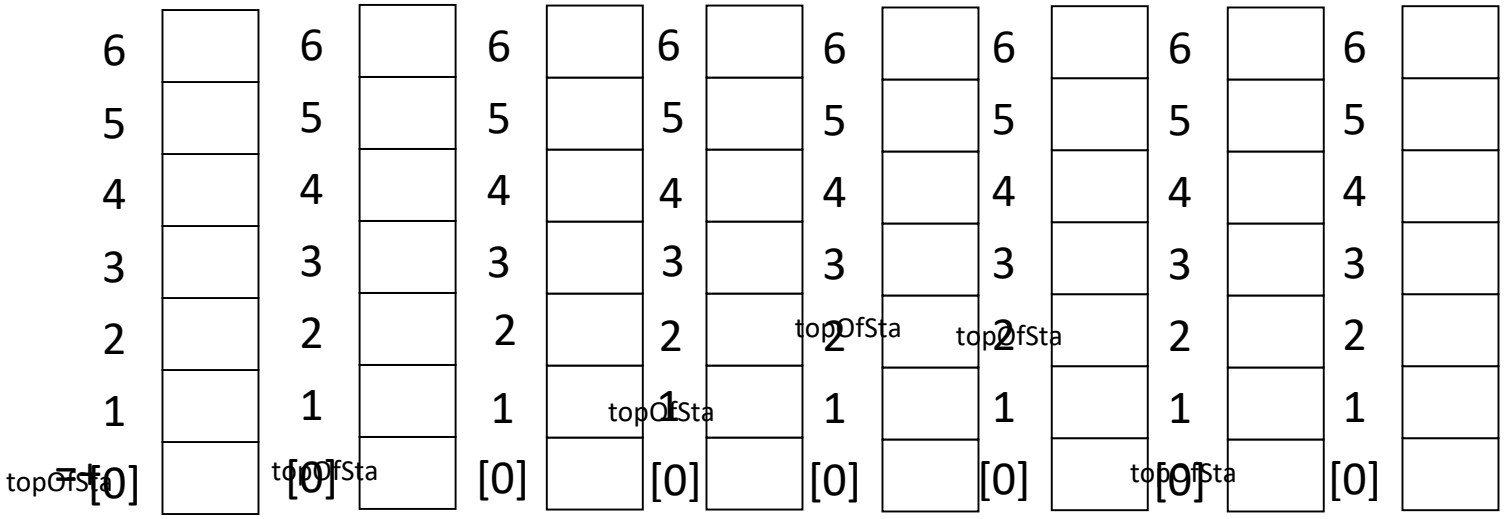
Algorithm:

- use a stack
- a number is seen → push onto stack
- an operator is seen → pop two values from stack
  apply operation
  push the result onto stack

Infix to Postfix Conversion (read postfix.pdf in google classroom)

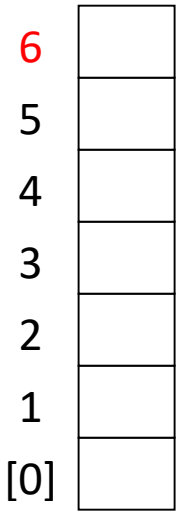
a + b * c + (d* e + f) * g

/



Output

/



For Example 4: Convert infix expression to postfix expression and evaluate that postfix expression.

(a+(b*c))+(((d*e)+f)*g)